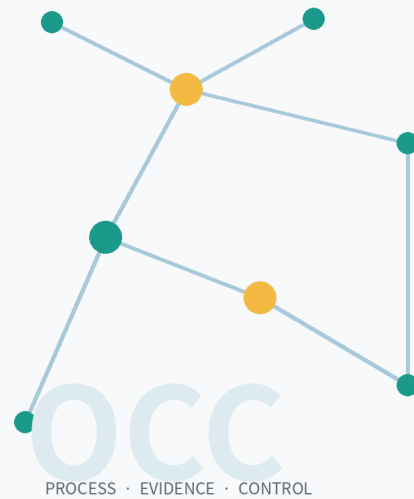


项目企划书

创序 OCC

高复杂度流程智能运营控制系统

从嵌入式开发教学的真实部署出发，
验证面向小微初创企业的虚拟运营控制中心



Innorder OCC Software Specification and Database Design

Document Language: English

Baseline: 16-week MVP and domain expansion

System:
Innorder OCC

Document version:
1.0

Prepared: 2026-07-28

Status: Design baseline / pre-implementation specification

Innorder OCC: Software Specification and Database Design

Innorder OCC Project Documentation

Abstract

This specification consolidates the Innorder OCC project proposal, approved architecture design, database design, eight executable PostgreSQL/Flyway migrations, and their contract tests. It defines product scope, functional requirements, architecture boundaries, domain semantics, critical transactions, security controls, non-functional requirements, and the physical database design. Innorder OCC separates a deterministic workflow control plane from a probabilistic intelligence plane: Flowable and OCC Core own verifiable state, authorization, evidence, and audit; the AI Service performs understanding, retrieval, planning, verification, and risk recommendation without directly advancing critical workflow facts. The document explicitly distinguishes constraints enforced by current DDL, responsibilities left to application code, and gates that remain to be validated before production use.

Contents

I	Purpose and Source Baseline	3
I-A	Purpose	3
I-B	Authoritative sources	3
I-C	Terminology	3
II	Product Scope	3
II-A	Objectives	3
II-B	MVP exclusions	3
II-C	Actors and use cases	3
III	Architecture Specification	4
III-A	System structure	4
III-B	Architectural invariants	5
III-C	Core module boundaries	5
IV	Functional Requirements	5
IV-A	Identity and authorization	5
IV-B	Domain packages and workflow	6
IV-C	Evidence, risk, and resources	6
IV-D	AI, RAG, and collaboration	6
V	Domain Model and Lifecycles	8
V-A	Aggregate boundaries	8
V-B	Key lifecycles	8
VI	Critical Interactions and Transactions	9
VI-A	Authorization-sensitive command	9
VI-B	Evidence submission and review	10
VII	API and Event Contracts	10
VIII	Security, Privacy, and Governance	10
IX	Non-functional Requirements	11
IX-A	Reliability and degradation	11
IX-B	Performance baseline	11
IX-C	Observability	11

X	Database Design	11
X-A	Platform and scale	11
X-B	Conventions	12
X-C	Schema and table inventory	13
X-D	Unified entity and shared keys	13
X-E	Extensible business data	14
X-F	Workflow, evidence, and resource integrity	14
X-G	AI and RAG model	15
X-H	Indexes, partitions, and immutability	15
X-I	Database account boundary	15
XI	Verification and Acceptance	16
XI-A	Existing evidence	16
XI-B	Release gates	16
XI-C	Pilot outcomes	16
XII	Implementation Gaps and Decisions	16
XIII	Project Document Summary	17
XIV	Conclusion	17
	References	17

I. Purpose and Source Baseline

A. Purpose

This document is the implementation, database review, interface design, test, and private-deployment baseline for the MVP. It is not a marketing document and does not present planned application components as completed software. The current repository contains a substantial database implementation and tests, but no Electron, OCC Core, AI Service, OPA bundle, or Docker Compose application implementation.

B. Authoritative sources

Source	Status	Use in this specification
Project Proposal V1.0	Product intent	Defines the embedded-development teaching pilot, product value, 16-week roadmap, success metrics, and responsibility boundary.
Technology and Architecture Design	Approved design	Defines technology choices, service boundaries, transaction/event semantics, degradation, security, and performance targets.
Database Design	Approved design	Defines schema ownership, data semantics, authorization, AI/RAG structures, and governance principles.
Migrations V001–V008	Executable implementation	Authoritative for objects, columns, constraints, triggers, indexes, and current partitions.
Database tests and README	Verification evidence	Identify tested contracts and limitations of the PGlite validation environment.

C. Terminology

OCC means operations control center. A domain package is a versioned release unit containing BPMN, DMN, forms, roles, evidence rules, risk rules, knowledge sources, and UI metadata. A business fact is authoritative state that can alter a process, task, evidence, or approval result. An AI recommendation is not a fact until accepted through an ordinary authorized command. A shared authorization entity is a principal or resource registered in authz.entity and available to the common relationship graph and OPA policy decision.

II. Product Scope

A. Objectives

- 1) Execute complex domain workflows as standard BPMN with deterministic rules, versioned forms, and evidence requirements.
- 2) Maintain actionable tasks, blockers, waiting-period work, and risk status for every participant.
- 3) Make every critical state transition attributable, authorized, evidenced, versioned, and auditable.
- 4) Run a private instance per customer and integrate with customer OIDC, object storage, Kafka, Redis, and model services.
- 5) Add domains by publishing packages instead of forking core services.
- 6) Preserve core correctness when AI, Kafka, or Redis is unavailable.

B. MVP exclusions

The MVP excludes multi-tenancy within one instance, offline writes and conflict merging, event sourcing, a custom BPMN engine, Kubernetes and service mesh, irreversible submissions to government or third-party systems, and AI-only final grading or high-liability compliance decisions. It also excludes a general cross-industry compliance database, a resource marketplace, and five separately deployed Agent services.

C. Actors and use cases

Actor	Goal	Principal use cases
Participant / student	Know the next valid action and avoid idle time	View tasks and route; submit evidence; inspect blockers; reserve resources; receive cited guidance.

Actor	Goal	Principal use cases
Process owner / teacher	Spend attention only where judgment is required	Monitor radar; review evidence; return or conditionally accept; handle risks; authorize exceptions; inspect bottlenecks.
Domain modeler	Maintain releasable process assets	Edit BPMN, DMN, forms, evidence, risk, and role definitions; validate, compare, and publish packages.
Administrator	Maintain security and operational baseline	Manage local/OIDC identities, relationships, policy, model providers, retention, deployment, and audit access.
Resource manager	Control scarce capacity	Maintain equipment, rooms, expert time, capacity, and reservation conflicts.
AI Service	Produce governed recommendations	Consume events; retrieve authorized knowledge; call granted tools; record runs, citations, and recommendations.

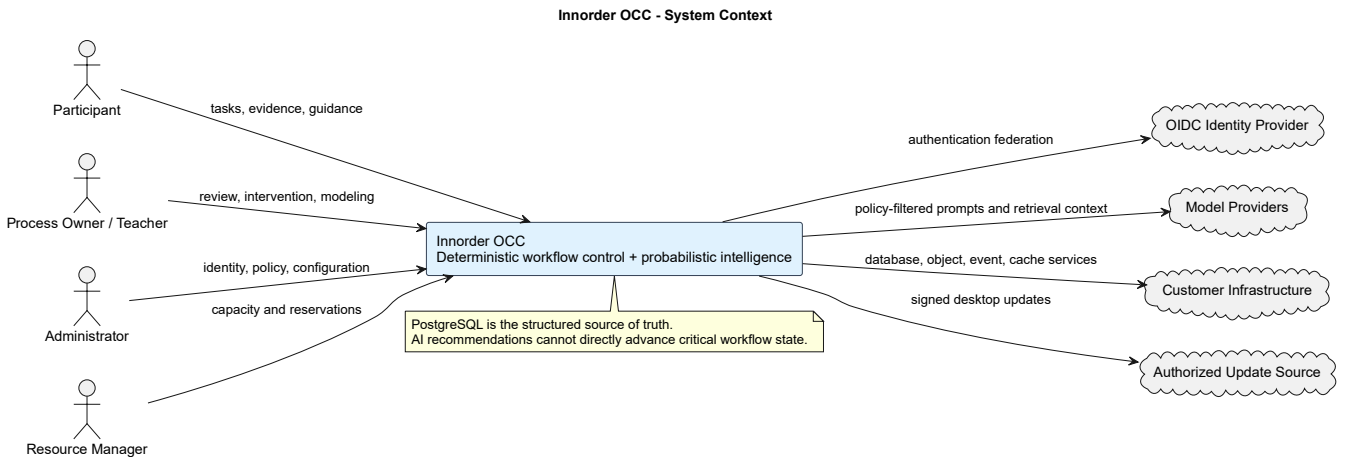


Fig. 1. System context and external trust boundaries.

III. Architecture Specification

A. System structure

The target system comprises an Electron desktop client, a Kotlin/Spring Boot modular-monolith Core with embedded Flowable, a separate Node.js/Fastify/LangChain.js AI Service, and PostgreSQL, OPA, Kafka, Redis, and S3-compatible storage. Each customer receives a private deployment. Electron uses domain APIs rather than native Flowable APIs. The AI Service accesses restricted Core APIs, authorized events, and AI-owned database objects. Event consumers cannot directly advance critical Flowable state.

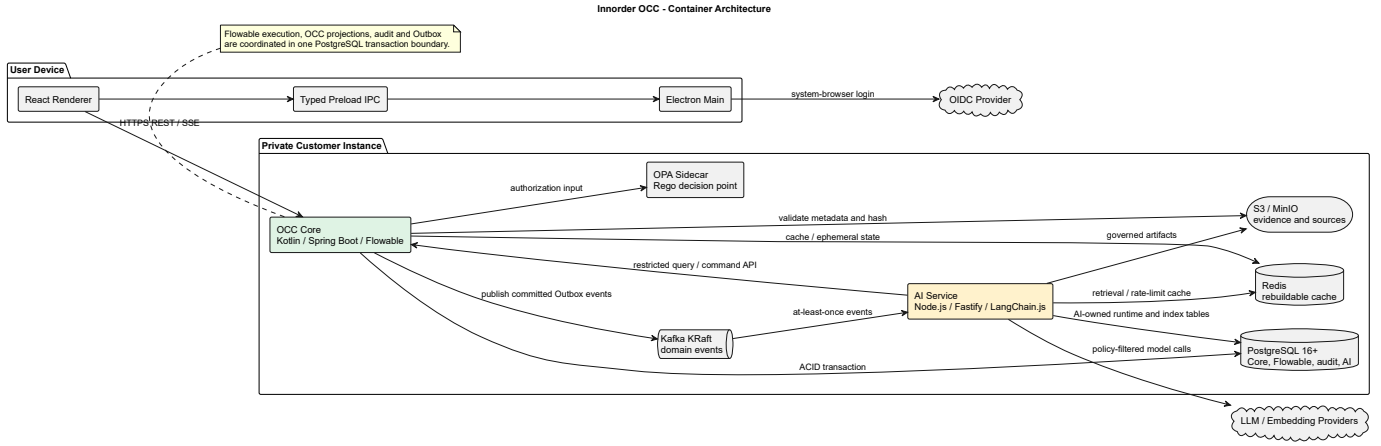


Fig. 2. Container architecture and synchronous/asynchronous communication.

B. Architectural invariants

- 1) PostgreSQL is the sole authority for structured business facts. Kafka is not a fact store, and Redis contents must be disposable and reconstructable.
- 2) Core coordinates Flowable changes, OCC projections, authorization relations, audit, and Outbox records in one database transaction.
- 3) An object becomes valid evidence only after Core verifies existence, type, size, and SHA-256 and commits evidence metadata.
- 4) OPA is a stateless decision point; PostgreSQL owns identities, direct relations, authorized attributes, and policy-release metadata.
- 5) Every applicable DENY overrides ALLOW. No match, OPA failure, or revision mismatch results in denial.
- 6) AI output remains a recommendation or human queue item and cannot bypass BPMN, DMN, evidence requirements, or approval gates.
- 7) Domain packages cannot contain arbitrary executable code, DDL, or secrets. Service tasks reference preregistered controlled capabilities.

C. Core module boundaries

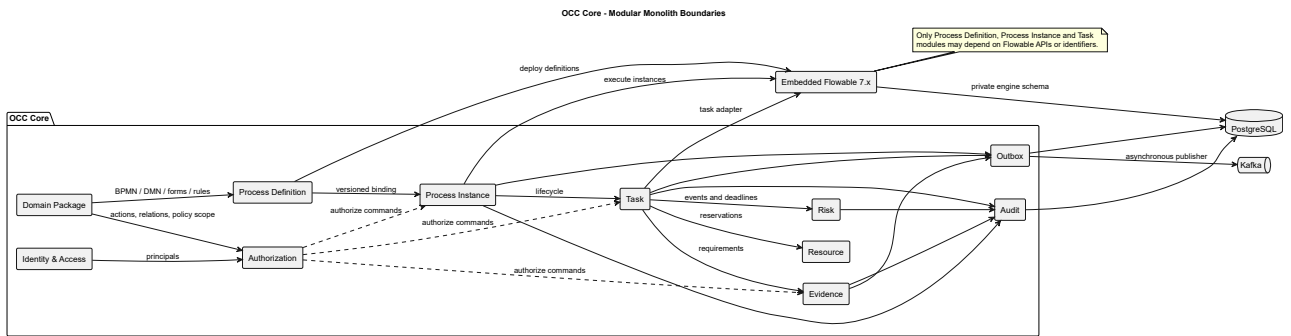


Fig. 3. OCC Core modular-monolith boundaries and Flowable encapsulation.

Only Process Definition, Process Instance, and Task modules may depend on Flowable APIs or engine identifiers. Other modules collaborate through internal domain interfaces and never query private Flowable tables. Identity & Access owns authenticating principals; Authorization constructs transaction-snapshot policy input; Domain Package owns versioned assets; Evidence, Risk, Resource, Audit, and Outbox own their corresponding facts.

IV. Functional Requirements

A. Identity and authorization

ID	Requirement	Enforcement
FR-IAM-01	The system shall support local user accounts and OIDC identity mappings. Groups and roles shall not authenticate.	Core
FR-IAM-02	Every authorizable principal and resource shall use a stable typed entity identity and optimistic row version.	DDL
FR-AZ-01	An authorization-sensitive write shall lock the authorization revision, load direct relations and the active release in the same transaction snapshot, and call local OPA.	Core/test
FR-AZ-02	Relationship writes shall validate published definitions, endpoint types, cardinality, validity, and optional acyclicity.	DDL
FR-AZ-03	A policy release shall atomically combine PLATFORM, DOMAIN, and CUSTOMER bundle versions, include a platform bundle, and be globally unique while ACTIVE.	DDL/Core
FR-AZ-04	Allowed write decisions shall be audited in the business transaction; denied decisions shall be appended in a separate transaction.	Core

B. Domain packages and workflow

ID	Requirement	Enforcement
FR-DP-01	A package version shall describe its manifest, BPMN/DMN references, JSON Schema forms, actions, relations, evidence, risk, and knowledge configuration.	Catalog/API
FR-DP-02	Package versions shall follow DRAFT, VALIDATED, PUBLISHED, and DEPRECATED states. Published content shall have a SHA-256 digest and be immutable.	DDL
FR-DP-03	Publication shall validate syntax, references, forbidden elements, permissions, regression scenarios, and impact on running instances.	Core
FR-WF-01	Every process instance shall explicitly bind a package version and Flowable definition binding; a new release shall not implicitly migrate it.	DDL/Core
FR-WF-02	State-changing commands shall use an idempotency key and expectedVersion. Hash mismatch or optimistic-lock conflict shall return HTTP 409.	Core
FR-WF-03	Human tasks shall support creation, claim, completion, cancellation, and failure; assignment and candidacy shall be represented as relationships.	Core/API
FR-WF-04	Flowable state, OCC projections, audit records, and Outbox events shall commit atomically.	Integration gate

C. Evidence, risk, and resources

ID	Requirement	Enforcement
FR-EV-01	The client shall upload through a short-lived session; Core shall create an immutable evidence version only after object verification.	DDL/Core
FR-EV-02	Evidence shall bind at least a task or business object and satisfy the applicable type, size, count, and acceptance requirement.	DDL/Core
FR-EV-03	Reviews shall be append-only and support ACCEPTED, REJECTED, and CONDITIONAL decisions with reviewer, reason, and conditions.	DDL
FR-RI-01	A risk shall preserve rule, target, INFO/YELLOW/RED severity, state, confidence, reason, deadline, and resolution time.	DDL
FR-RS-01	Active exclusive reservations shall not overlap for a resource. Capacity reservations shall be checked while holding a resource-row lock.	DDL/Core

D. AI, RAG, and collaboration

ID	Requirement	Enforcement
FR-AI-01	The AI Service shall support planning, tutoring, verification, risk, and supervision roles with shared governed tools, traces, and evaluation.	AI Service

ID	Requirement	Enforcement
FR-AI-02	A model profile shall declare capability, data policy, timeout, rate limit, and cost. Missing required capability shall fail closed.	DDL/AI
FR-AI-03	Tool calls shall use structured schemas, explicit grants and constraints, a required action, and a referenced OPA decision log.	DDL/contract
FR-RAG-01	Retrieval shall authorize document IDs before full-text/vector search and reranking within that candidate set.	Security test
FR-RAG-02	At most one embedding space may be ACTIVE. A replacement shall switch atomically only after coverage and retrieval evaluation gates.	DDL/AI
FR-AI-04	A recommendation shall record its run, target, structured payload, confidence, review state, and versioned citations. Acceptance remains an ordinary authorized command.	DDL/Core
FR-AI-05	Messages, prompts, knowledge, citations, tool calls, and evaluation versions shall remain traceable, and generated content shall be marked.	DDL/UI

V. Domain Model and Lifecycles

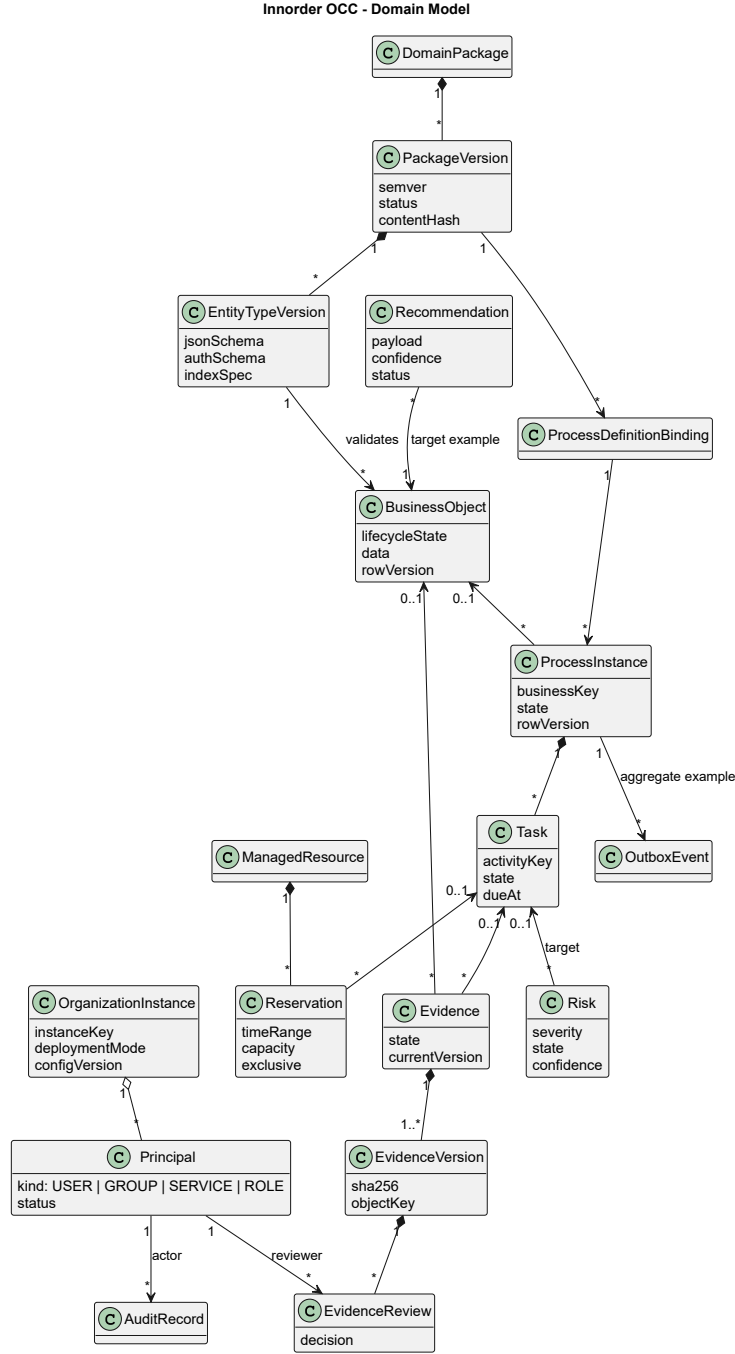


Fig. 4. Stable domain concepts and aggregate relationships.

A. Aggregate boundaries

A domain package and its versioned definitions form a publication aggregate. A business object is the extensible domain-data aggregate. Process instances and tasks are OCC-authorizable projections of Flowable state. Evidence consists of a mutable head, immutable versions, and append-only reviews. Risks and reservations represent intervention and resource-use aggregates. An AI run is execution trace data, while a recommendation is an authorizable and reviewable business proposal.

B. Key lifecycles

Object	States and invariants
Package Version	DRAFT → VALIDATED → PUBLISHED → DEPRECATED; published content is immutable.
Process Instance	RUNNING/SUSPENDED → COMPLETED/CANCELLED/FAILED; terminal state requires ended_at.
Task Projection	CREATED → CLAIMED → COMPLETED, or CANCELLED/FAILED; completion requires completed_at.
Evidence	PENDING → SUBMITTED → ACCEPTED/REJECTED, optionally ARCHIVED; current version uses a deferred FK.
Risk	OPEN → ACKNOWLEDGED → RESOLVED/DISMISSED; RESOLVED requires resolved_at.
Policy Release	Inserted only as STAGED, then ACTIVE/RETIRED/FAILED; ACTIVE is globally unique.
Embedding Space	BUILDING → ACTIVE → RETIRED, or FAILED; ACTIVE is globally unique.
Recommendation	PROPOSED → ACCEPTED/REJECTED/EXPIRED/WITHDRAWN.

VI. Critical Interactions and Transactions

A. Authorization-sensitive command

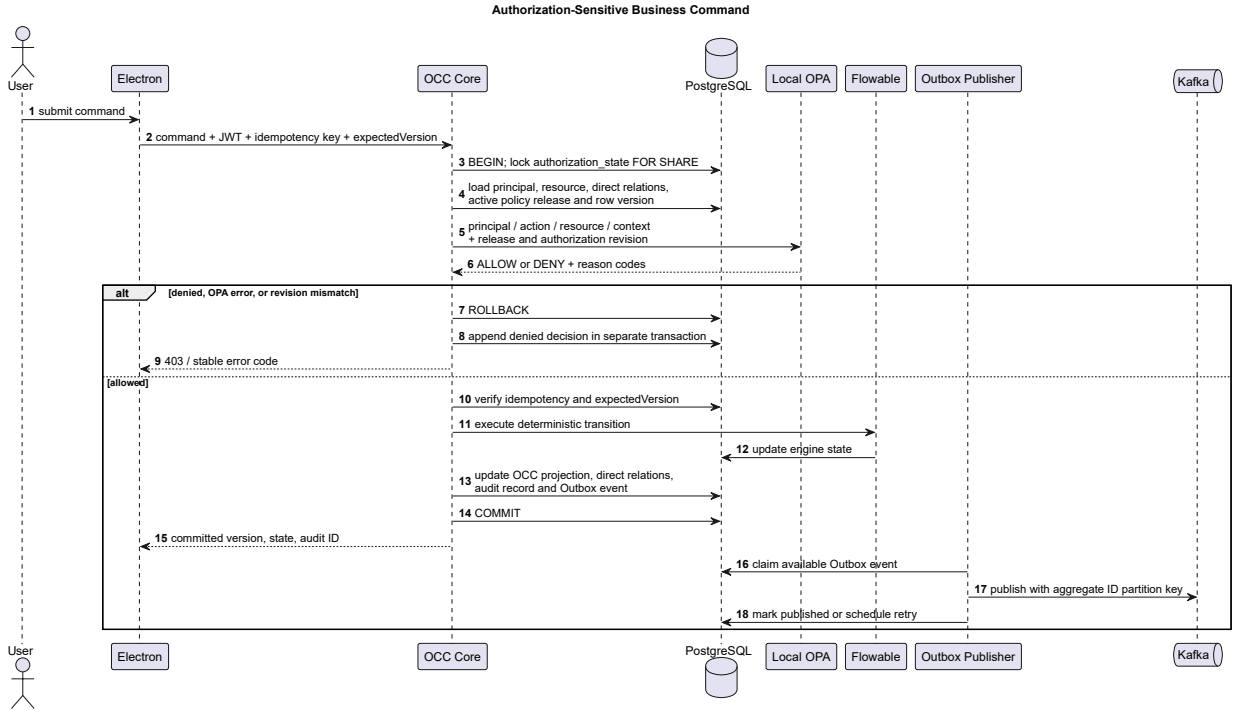


Fig. 5. Authorization, optimistic locking, Flowable, audit, and Outbox order.

A normal business write obtains a shared lock on `authz.authorization_state`. A relation, policy release, authorization attribute, or entity authorization-state change obtains an exclusive lock and increments the revision. Core sends the locked revision and policy release to OPA and rejects mismatched results. Kafka delivery is at least once; consumers deduplicate by event ID and partition by aggregate ID to preserve aggregate-local order.

B. Evidence submission and review

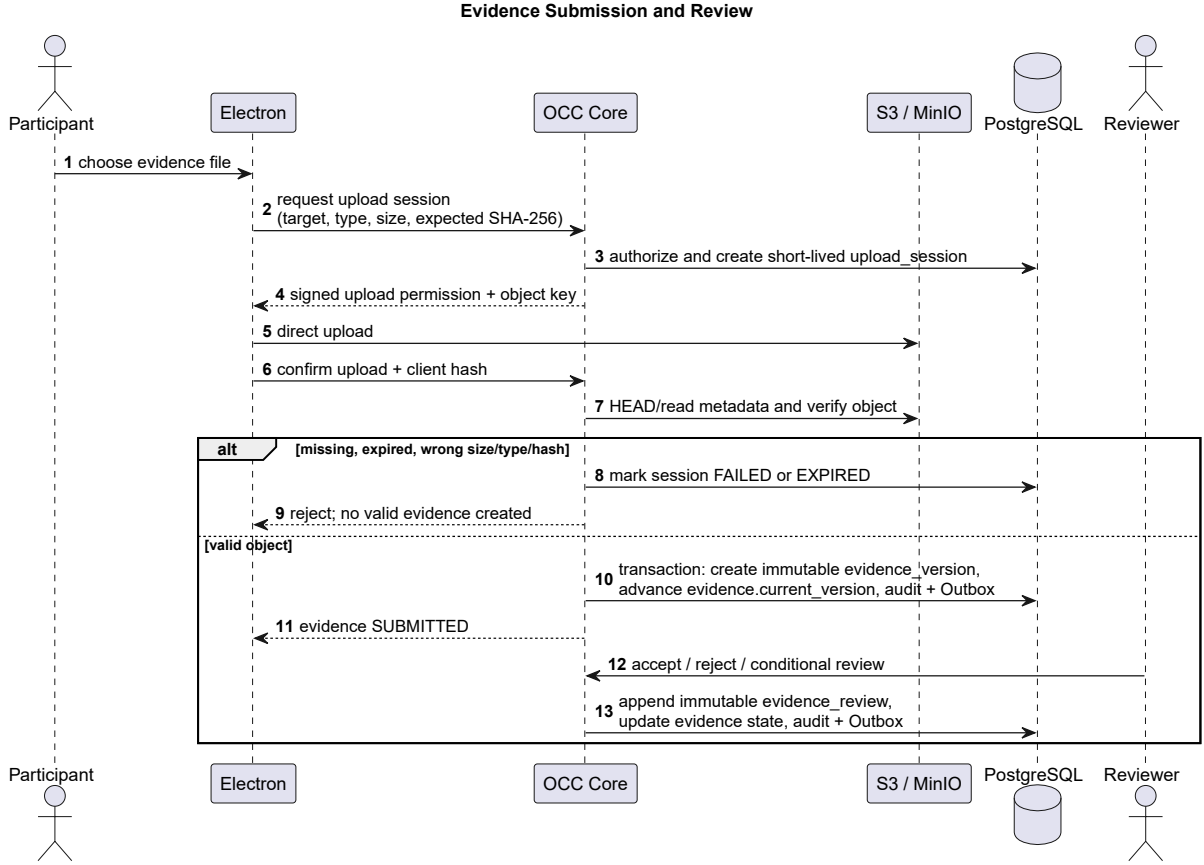


Fig. 6. Evidence lifecycle from temporary object upload to review.

VII. API and Event Contracts

REST/OpenAPI is the synchronous contract source for Electron, Core, and AI. Commands carry JWT, Idempotency-Key, expectedVersion, and correlation ID. Errors use Problem Details with a stable code, retryability, correlation ID, and user-readable detail. Domain APIs must not expose native Flowable table names, entities, or internal state codes. SSE carries real-time notifications, but reliable notifications are persisted before delivery.

Kafka JSON envelopes include event ID, schema version, customer instance ID, aggregate type/ID/version, event type, occurrence time, correlation ID, and data classification. CI shall enforce declared compatibility policy. PostgreSQL-to-Kafka exactly-once delivery is explicitly not promised; the model is transactional Outbox, at-least-once publication, aggregate-local ordering, and idempotent consumption.

VIII. Security, Privacy, and Governance

- 1) The Electron renderer enables sandbox, context isolation, and CSP and disables Node integration. The main process protects tokens in OS secure storage.
- 2) Services use separate identities, TLS, and least-privilege database accounts. Only external secret references are persisted.
- 3) Platform DENY rules cannot be overridden. Policy publication requires compilation, tests, approval, signing, OPA load confirmation, and atomic activation.
- 4) Authorization attributes are restricted by auth-schema allowlists. Provider-specific data policies default-deny model-bound fields and classifications.
- 5) Retrieved content is untrusted input and cannot expand system instructions, tool grants, authorization, or approval rules.
- 6) Audit, evidence versions, policy releases, document versions, and messages are protected through append-only or immutable semantics. Retention and legal hold govern deletion.
- 7) AI output retains model, prompt, package, knowledge, citations, tool calls, cost, and trace metadata.

IX. Non-functional Requirements

A. Reliability and degradation

Kafka failure delays consumers while Core commits and retains Outbox records. Redis failure causes PostgreSQL fallback. AI timeout, rate limiting, malformed output, or missing citations rejects the recommendation or escalates to a person without blocking core commands. Object-store failure cannot create valid evidence. If Core or PostgreSQL is unavailable, Electron offers read-only cached data and disables all writes.

B. Performance baseline

On a recorded representative private-server configuration, 100 concurrent sessions shall meet: cache-hit query p95 ≤ 200 ms; PostgreSQL fallback p95 ≤ 500 ms; state-changing command excluding file and AI p95 ≤ 700 ms; committed domain event to consumer p95 ≤ 2 s. Retry and concurrency conflict shall cause no lost or duplicate transition. AI first-token latency, completion latency, failure rate, and cost are reported separately from the Core API SLA.

C. Observability

All services use structured logs and a common correlation ID. OpenTelemetry covers API, database, Kafka, and model calls. Required metrics include request latency/error rate, Flowable jobs, Outbox backlog, consumer lag, dead letters, Redis hit rate, object upload failure, OPA latency/error, AI token/cost/latency, partition health, and backup status.

X. Database Design

A. Platform and scale

The database baseline is PostgreSQL 16+, UTF-8, with vector and btree_gist. Current migrations create eight schemas and 70 logical tables plus two default partitions. The flowable schema namespace exists, but vendor engine tables are created and upgraded only by the pinned Flowable release.

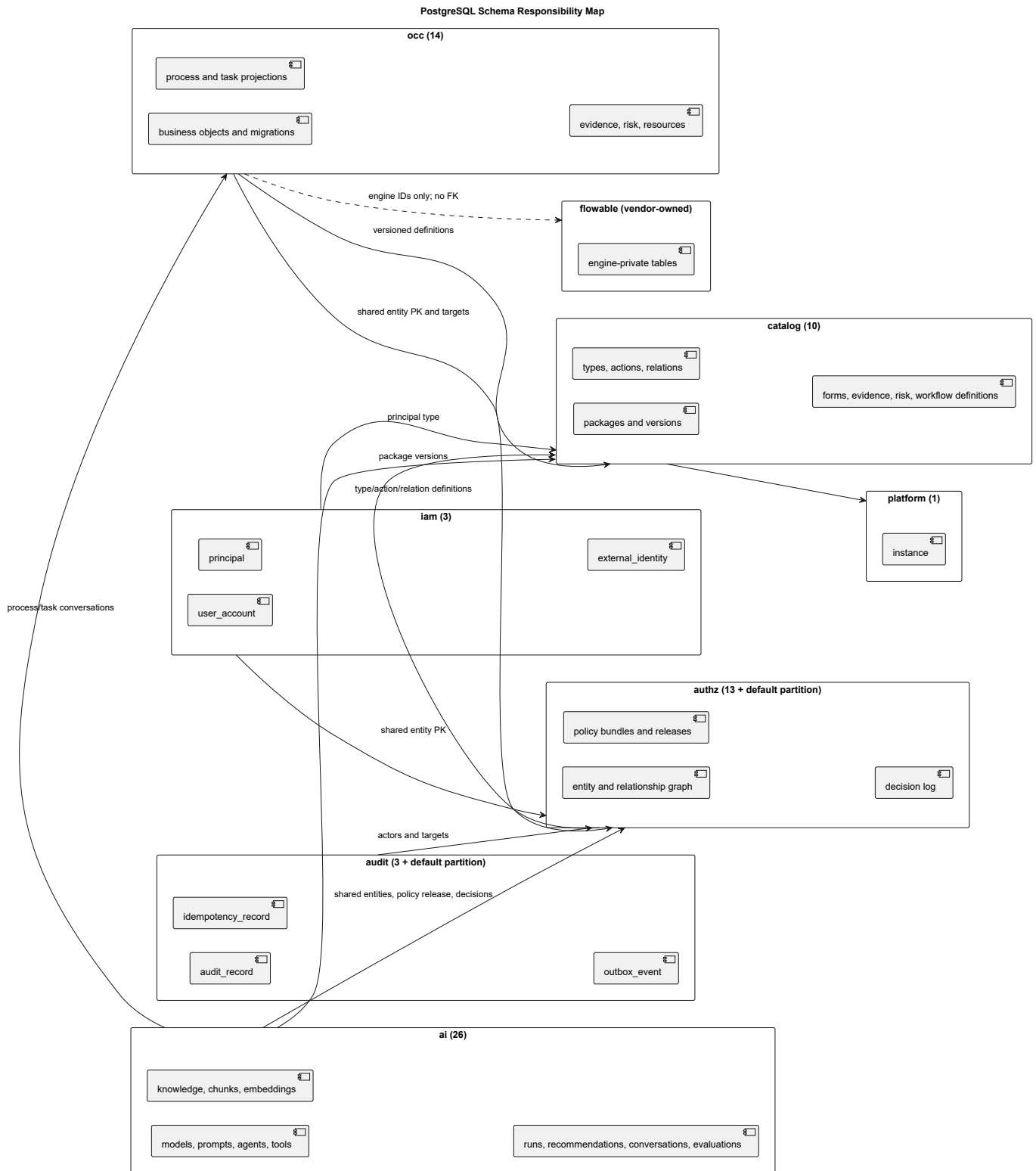


Fig. 7. PostgreSQL schema responsibilities and dependency direction.

B. Conventions

Applications generate UUIDv7 keys. Timestamps are UTC timestamptz; monetary values and cost use numeric; extensible states use text plus CHECK rather than PostgreSQL enums. JSON object columns use platform.is_json_object. Mutable aggregates use row_version; update triggers refresh updated_at and increment the version. Delete defaults to RESTRICT, with CASCADE limited to draft children, rebuildable projections, and tightly owned runtime children.

C. Schema and table inventory

Schema	Count	Tables
platform	1	instance
catalog	10	domain_package, package_version, entity_type, entity_type_version, action_definition, relation_definition, form_definition, evidence_requirement, risk_rule_definition, workflow_definition
iam	3	principal, user_account, external_identity
authz	13	entity, authorization_state, relationship, relationship_closure, policy_bundle, policy_bundle_version, policy_module, policy_test_case, policy_approval, policy_binding, policy_release, policy_release_item, decision_log (plus decision_log_default)
occ	14	business_object, object_index_value, data_migration, data_migration_item, process_definition_binding, process_instance, task_projection, upload_session, evidence, evidence_version, evidence_review, risk, managed_resource, resource_reservation
audit	3	audit_record, idempotency_record, outbox_event (plus audit_record_default)
ai	26	model_provider, model_profile, prompt_template, prompt_template_version, agent_definition, agent_definition_version, tool_definition, agent_tool_grant, knowledge_source, knowledge_document, knowledge_document_version, knowledge_chunk, embedding_space, chunk_embedding, ai_run, ai_run_artifact, tool_call, recommendation, recommendation_citation, conversation, message, evaluation_dataset, evaluation_dataset_version, evaluation_case, evaluation_run, evaluation_result
flowable	0	Vendor-private tables are Flowable-owned; OCC stores engine IDs and defines no cross-boundary foreign keys.

D. Unified entity and shared keys

authz.entity is the authorization supertype. iam.principal, six OCC entities (business object, process instance, task, evidence, risk, and managed resource), and five AI entities (provider, knowledge source/document, recommendation, and conversation) share its primary key. Composite foreign keys align stable entity types with their versions and align a business object with the entity version. Direct relationship facts have a single source in authz.relationship; closure is a rebuildable candidate projection and never an authoritative ALLOW fact.

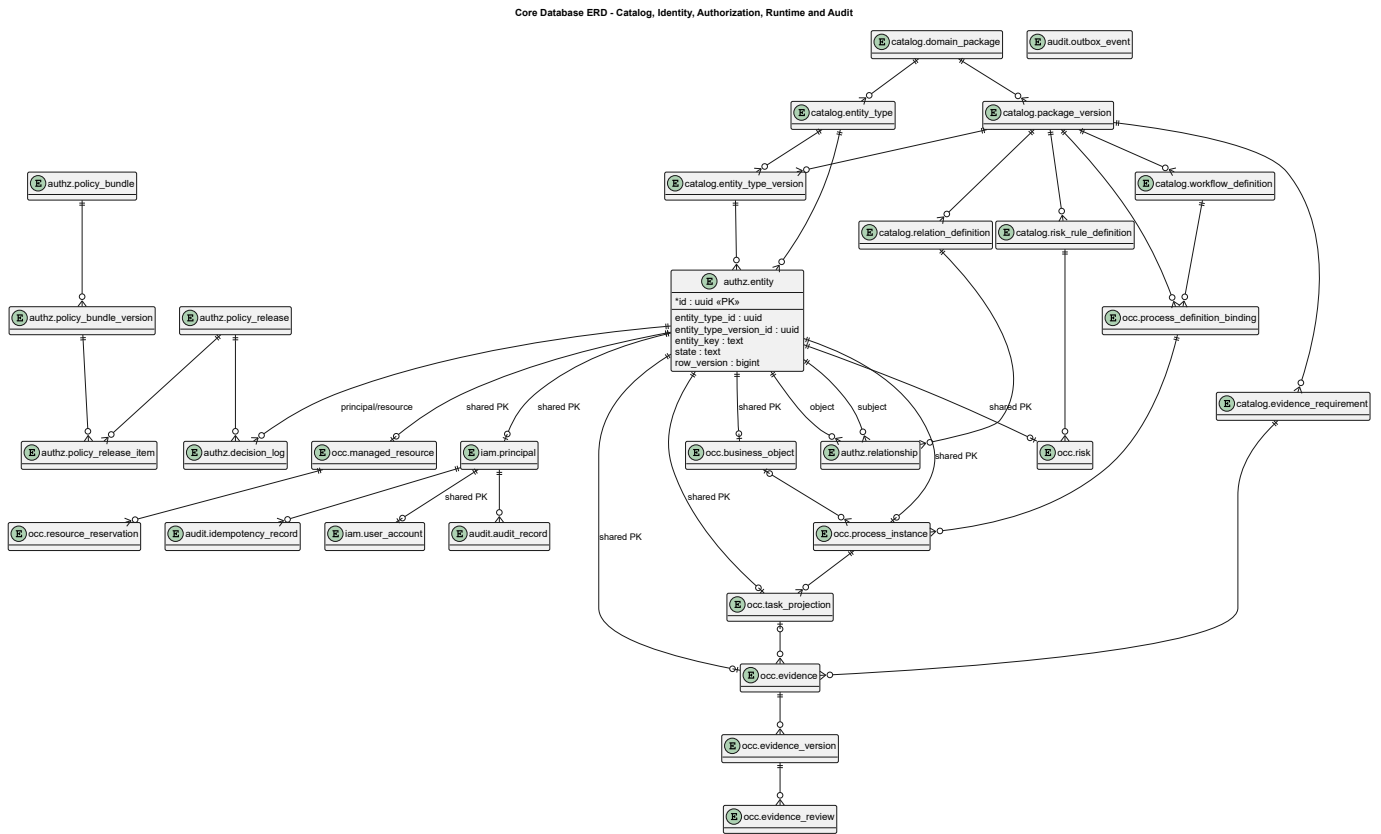


Fig. 8. Core ERD; selected fields and secondary definition tables are omitted for readability.

E. Extensible business data

`occ.business_object.data` stores domain fields validated against JSON Schema and remains bound to its creation-time entity-type version. `object_index_value` is a rebuildable typed projection for TEXT, NUMBER, TIMESTAMP, and BOOLEAN filtering, not an EAV source of truth. Explicit data migration and item records track source/target hashes and status; an object changes version only after its item succeeds.

F. Workflow, evidence, and resource integrity

Flowable deployment, definition, instance, and task identifiers are unique but have no FK to vendor tables. Evidence versions preserve object key, SHA-256, MIME type, and size as immutable facts; a deferred composite FK guarantees current version belongs to the same evidence. A GiST exclusion constraint prevents overlapping active exclusive reservations. Core remains responsible for aggregate capacity checks while locking the resource row.

G. AI and RAG model

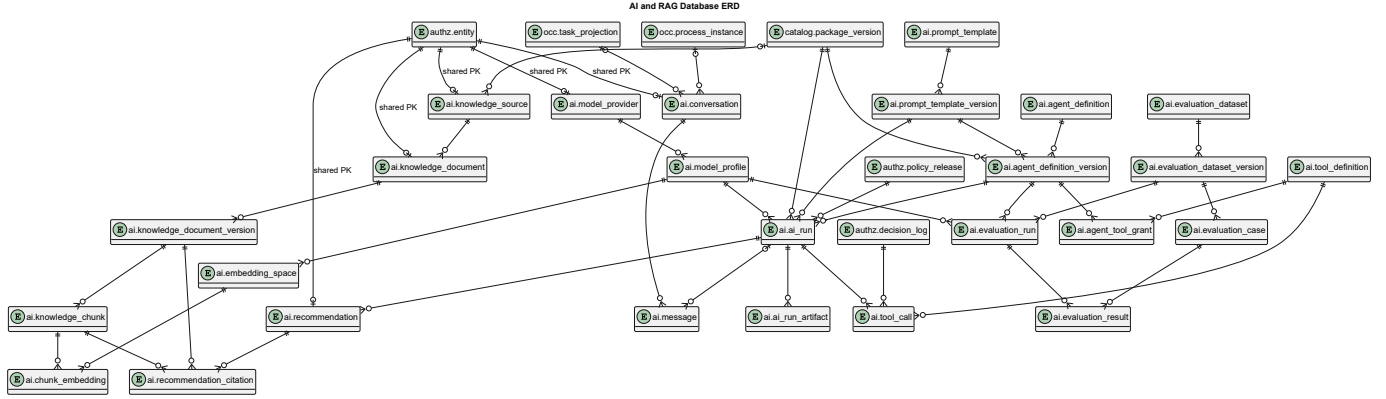


Fig. 9. AI, RAG, recommendation, conversation, and evaluation ERD.

Knowledge sources and documents are authorizable entities; chunks inherit document authorization. Full-text search uses a generated tsvector and GIN. Vector data is list-partitioned by embedding space; a helper creates the per-space dimension CHECK and metric-specific COSINE/L2/INNER PRODUCT HNSW index. Tool calls reference partitioned decision logs by (ID, created_at). A composite citation FK proves that each cited chunk belongs to the declared document version.

H. Indexes, partitions, and immutability

Indexes cover entity type/state, bidirectional relationship traversal and validity, JSONB business data, typed projection values, task/evidence/risk queues, pending Outbox, knowledge full text, AI runs, and message chronology. Decision and audit logs are range-partitioned with writable default partitions. Chunk embeddings are list-partitioned by space. Operations must create monthly partitions ahead of time and move eligible default-partition rows during maintenance.

I. Database account boundary

The target deployment has migration, Core, AI, and read-only audit accounts. The AI account cannot read Flowable, credential, or private OCC data and cannot write IAM, authorization, or OCC facts. Current migrations create no roles, grants, restricted views, or RLS, so deployment SQL and privilege regression tests remain a mandatory delivery gate.

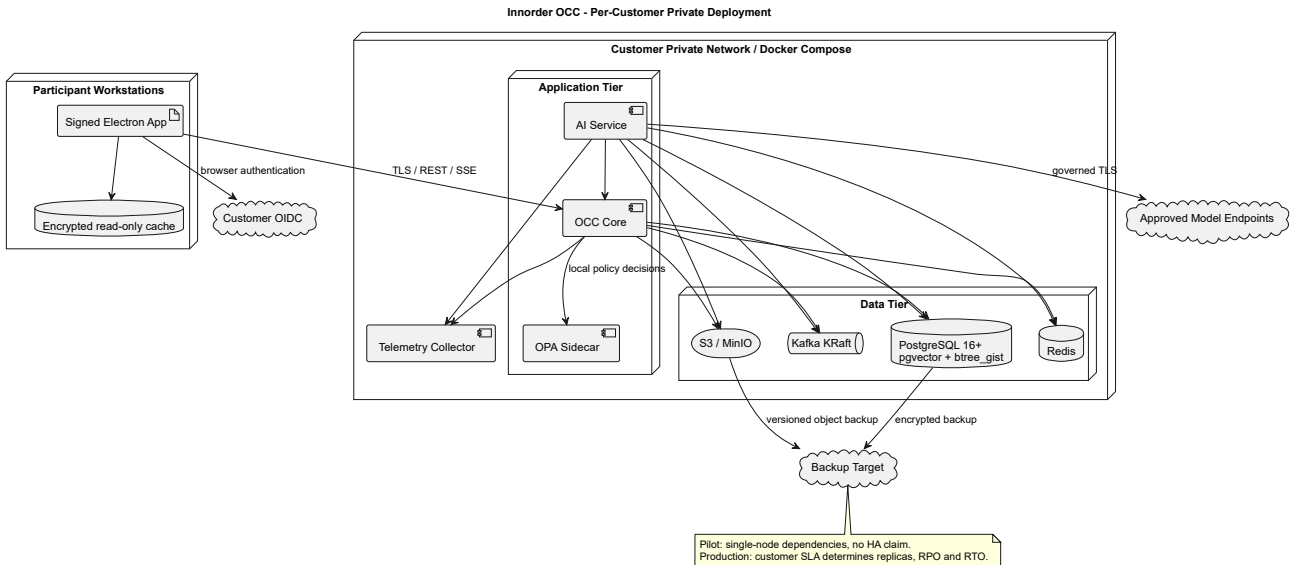


Fig. 10. Per-customer private deployment topology.

XI. Verification and Acceptance

A. Existing evidence

Node static tests verify migration order, absence of placeholders, schemas/extensions, critical tables, authorization/concurrency safeguards, trigger return patterns, cross-table versions, and immutable policy content. SQL contract tests exercise key objects and selected mutation constraints. The README records a successful clean PGLite 0.5.4 run, but PGLite used a compatibility domain in place of pgvector.

B. Release gates

- 1) Apply V001–V008 and all SQL contracts to real PostgreSQL 16 with pgvector.
- 2) Execute `ai.create_embedding_partition` and verify dimensions, operator class, and HNSW creation.
- 3) Use Testcontainers to prove Flowable/OCC/audit/Outbox atomicity, Kafka replay, Redis degradation, and object-upload semantics.
- 4) Test concurrent revocation, policy switch, relationship change, and business writes to exclude TOCTOU authorization bypass.
- 5) Test default deny, DENY precedence, non-overridable platform DENY, relationship cardinality, and cycle rejection.
- 6) Prove unauthorized documents never enter RAG candidates and test prompt injection, tool escalation, and uncited output.
- 7) Execute Electron security, end-to-end, read-only offline, installer signing, upgrade, and rollback tests.
- 8) Run k6 performance baselines and rehearse backup restore, policy rollback, partition archive, and object cleanup.

C. Pilot outcomes

For the first 10–20 student pilot: critical-node evidence completeness $\geq 95\%$; invalid-skip interception $\geq 90\%$; repetitive teacher follow-up time reduced $\geq 30\%$; valuable waiting-period task coverage $\geq 70\%$; zero missed known severe red risks; risk false-positive rate $\leq 20\%$; weekly student activity $\geq 80\%$; process changes effective within one business day.

XII. Implementation Gaps and Decisions

Topic	Current fact	Required disposition
Service credentials	Design allows USER/SERVICE authentication; DDL only has USER-only user_account.	Select OIDC client credentials, mTLS, or a controlled new persistence model before implementation.
AI run immutability	Design calls AI runs append-only; DDL has no immutable trigger for run, artifact, tool call, or evaluation records.	Define mutable lifecycle columns and freeze terminal records or add history.
Monthly partitions	DDL has only audit/decision default partitions, no maintenance helper; AI runs are unpartitioned.	Add operational migration and monitoring; partition AI logs if measured volume warrants it.
Least privilege	DDL defines no roles, grants, restricted views, or RLS.	Deliver deployment privilege SQL and regression tests.
Shared-PK subtype	Principal and business object types are checked; other shared-PK tables do not prove expected subtype.	Enforce in Core factories or database triggers and test it.
Cross-package consistency	Several workflow/binding/instance and agent/prompt/package pairs do not fully prove compatible package ownership.	Reject inconsistent references during publication and add composite FKs where practical.
Policy publication gates	DDL does not require all tests and approvals before status publication.	Core publication command must verify compilation, tests, and approval before transition.
Real pgvector	PGLite substituted a compatible type and did not verify actual HNSW behavior.	Treat real PostgreSQL/pgvector execution as a non-waivable production gate.

XIII. Project Document Summary

The proposal establishes the product thesis: begin with embedded-development education to validate an “people–task–evidence–resource–risk” control kernel in a low-liability, repeatable setting, using a 16-week, 10–20 participant pilot. The architecture design turns that thesis into per-customer private deployment, an Electron client for all users, embedded Flowable in a modular Core, a separate AI Service, three-layer OPA policy, Kafka Outbox events, and rebuildable Redis caches. The database design turns cross-domain extensibility into a versioned catalog, unified authorization entities, strong relational cores with JSONB domain data, workflow projections, immutable evidence, explicit data migration, and authorization-first RAG. The SQL migrations implement most of this data foundation and major integrity rules; application services, deployment privileges, real infrastructure integration, and several governance gates remain to be built and verified.

XIV. Conclusion

Innorder OCC places correctness in a deterministic Core and transactional database while constraining AI to traceable, rejectable, human-reviewable recommendations. The current database baseline is sufficient to begin the MVP for domain packages, authorization graph, process projections, evidence, risk, resources, Outbox, and AI/RAG. The next engineering priorities are a Flowable atomic-transaction prototype, OPA revision concurrency tests, database-account isolation, real pgvector verification, and the first teaching domain package, rather than expanding the number of Agents or industries.

References

- [1] Innorder OCC Project Proposal, V1.0, July 2026.
- [2] Innorder OCC Technology Selection and Architecture Design, July 28, 2026.
- [3] Innorder OCC Database Design, July 28, 2026.
- [4] PostgreSQL Global Development Group, “PostgreSQL Documentation,” <https://www.postgresql.org/docs/>.
- [5] Flowable, “Flowable Open Source Documentation,” <https://www.flowable.com/open-source/docs/>.
- [6] Open Policy Agent, “OPA Documentation,” <https://www.openpolicyagent.org/docs/latest/>.
- [7] pgvector, “Open-source vector similarity search for Postgres,” <https://github.com/pgvector/pgvector>.